SIMATS ENGINEERING
ASSESSMENT TOOL 1

NAME: K.Deepika

REG NO: 192371042
COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4303

CO3-AT1

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.
Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Answer:

Servlet Architecture, GET/POST and Servlet Life Cycle

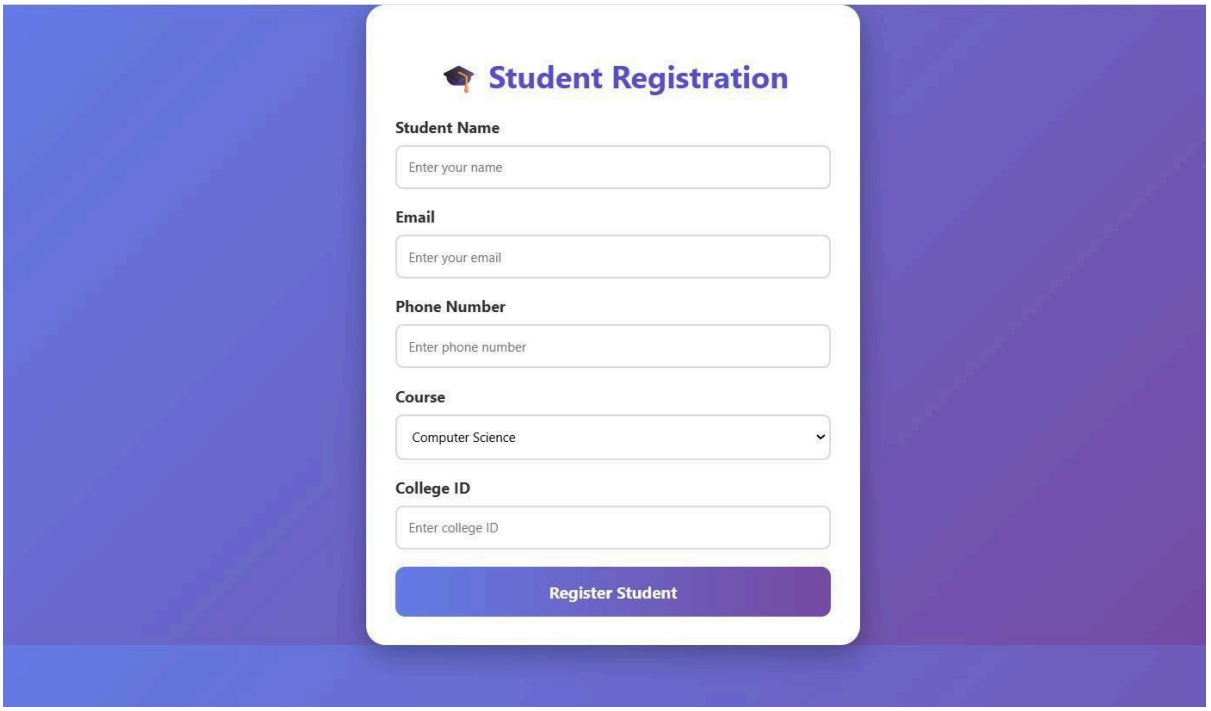
1. Servlet Architecture

A **Servlet** is a Java program that runs on a web server and handles requests from clients such as web browsers.

The basic architecture is:

Student
↓
Web Browser
↓ HTTP Request
Web Server / Servlet Container
↓
StudentRegistrationServlet
↓
Processing / Validation
↓
HTTP Response
↓
Web Browser

2. HTML Registration Form



3. Servlet Using GET and POST

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class StudentRegistrationServlet extends HttpServlet {

   protected void doGet(HttpServletRequest request,

                        HttpServletResponse response)
                        throws ServletException, IOException {

      response.setContentType("text/html");

      PrintWriter out = response.getWriter();

      String name = request.getParameter("name");
      String course = request.getParameter("course");

      out.println("<h2>GET Request</h2>");
      out.println("<p>Name: " + name + "</p>");
      out.println("<p>Course: " + course + "</p>");
   }
```

```
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
                          throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        out.println("<h2>Registration Successful</h2>");
        out.println("<p>Name: " + name + "</p>");
        out.println("<p>Email: " + email + "</p>");
        out.println("<p>Course: " + course + "</p>");
    }
}
```

4. Servlet Life Cycle

The servlet life cycle consists of three main methods:

1. init()
Called **only once** when the servlet is created.

```
public void init() {
    System.out.println("Servlet initialized");
}
```

Used for initialization tasks such as loading configuration.

2. service()
Called whenever the servlet receives a request.

Client Request

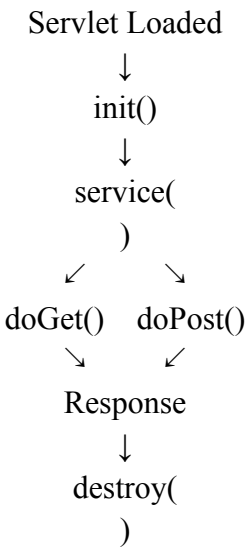
```
    ↓
  service()
    ↓
doGet() / doPost()
```

The service() method determines whether the request is GET, POST, etc., and calls the appropriate method.

3. destroy()
Called once before the servlet is removed from memory. public void destroy() {
   System.out.println("Servlet destroyed");
}

Used for cleanup operations.

Life Cycle Diagram



5. GET vs POST

GET	**POST**
Data is sent in URL	Data is sent in request body
Less suitable for sensitive data	More appropriate for sensitive data
Data can appear in browser history	Data is not normally displayed in URL
Suitable for retrieving data	Suitable for submitting data
Limited URL length	Can handle larger request data

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.
Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Answer:

Session Management Using HttpSession and Cookies

1. Session Management

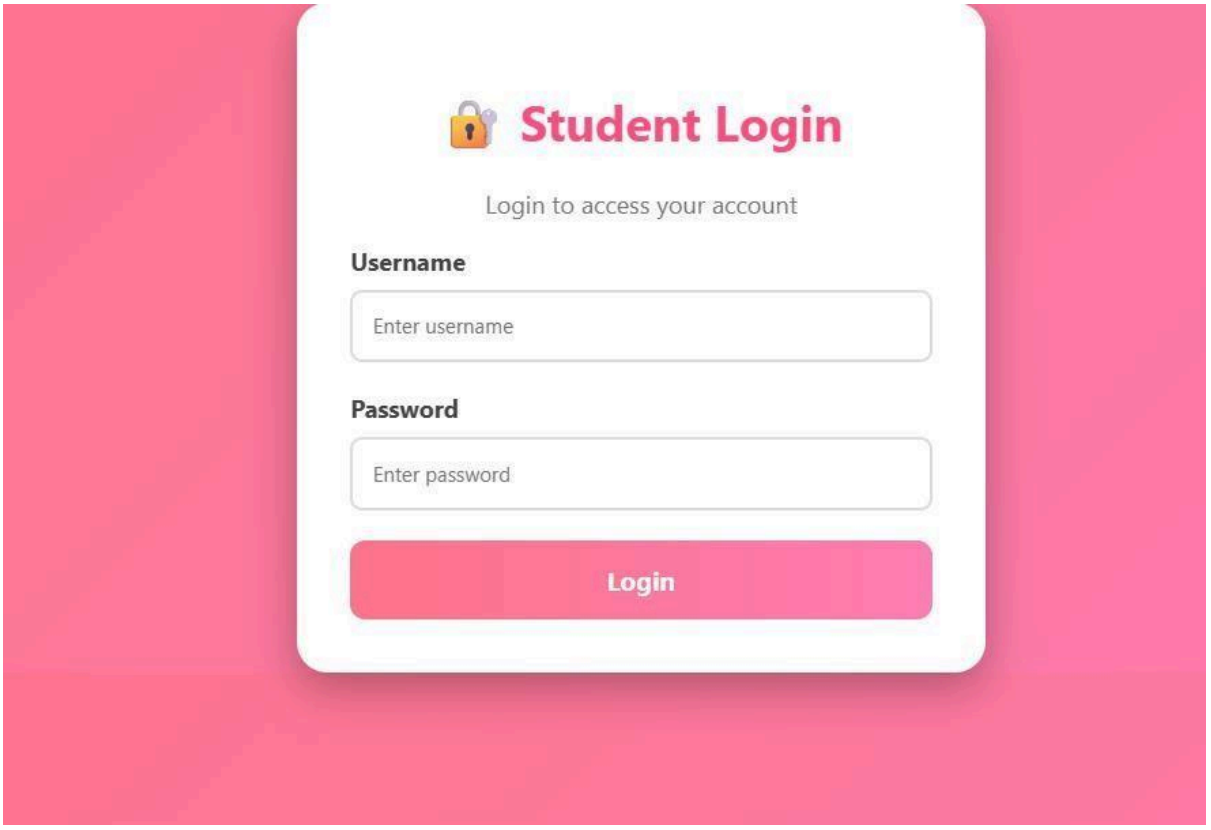
Session management allows a web application to remember a user across multiple HTTP requests.

For example:

Login
↓
Home Page
↓
Profile
↓
Product
s

The server needs to know that all these requests belong to the same user.

2. Login Servlet Using Http Session



3. Login Servlet Using HttpSession

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class LoginServlet extends HttpServlet {

   protected void doPost(HttpServletRequest request,

                         HttpServletResponse response)
                         throws ServletException, IOException {

      String username = request.getParameter("username");
      String password = request.getParameter("password");

      if (username.equals("admin") &&
         password.equals("1234")) {

         HttpSession session = request.getSession();

         session.setAttribute("username", username);

         response.sendRedirect("home");
      }
      else {
         response.setContentType("text/html");

         PrintWriter out = response.getWriter();

         out.println("<h2 style='color:red'>");
         out.println("❌ Invalid Username or Password");
         out.println("</h2>");
      }
   }
}
```

4. Home Servlet

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class HomeServlet extends HttpServlet {

   protected void doGet(HttpServletRequest request,
```

```
                HttpServletResponse response)
                throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        HttpSession session =
           request.getSession(false);
        if (session != null) {

           String username

           =

              (String) session.getAttribute("username");

           out.println("<html>");
           out.println("<body
style='font-family:Arial;text-align:center;background:#e0f7fa;padding:50px'>");

           out.println("<div
style='background:white;padding:40px;border-radius:20px;width:450px;margin:auto'>");

           out.println("<h1
           style='color:#0097a7'>"); out.println("🎉
           Welcome " + username);
           out.println("</h1>");

           out.println("<p>You are successfully logged in.</p>");

           out.println("<a href='logout'>Logout</a>");

           out.println("</div>");
           out.println("</body>");
           out.println("</html>");

        } else {

           out.println("<h2>Please Login First</h2>");
        }
     }
}
```

5. Logout Servlet

```
import java.io.*;
```

```
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class LogoutServlet extends HttpServlet {

   protected void doGet(HttpServletRequest request,

                        HttpServletResponse response)
                        throws IOException {

      HttpSession session =
         request.getSession(false);

      if (session != null) {
         session.invalidate();
      }

      response.sendRedirect("login.html");
   }
}
```

6. Cookies

A **cookie** is a small piece of data stored by the browser.

Create Cookie

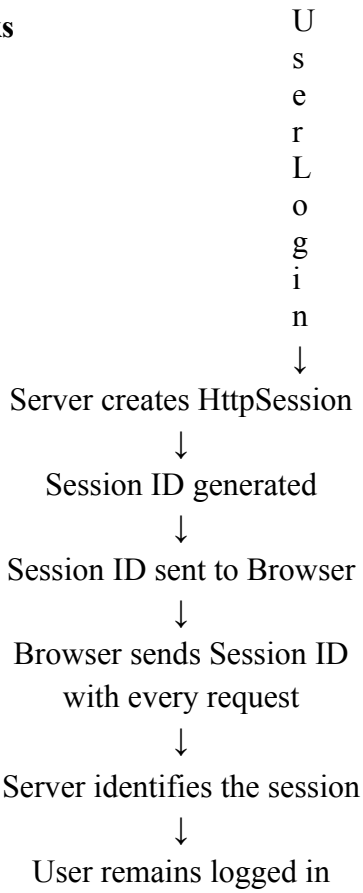
```
Cookie cookie =
   new Cookie("username", "Sravanthi");

cookie.setMaxAge(60 * 60);

response.addCookie(cookie);
```

Read Cookie

```
Cookie[] cookies = request.getCookies();

if (cookies != null) {

   for (Cookie cookie : cookies) {
      if (cookie.getName().equals("username")) {

         String username =

            cookie.getValue();
```

```
            System.out.println(username);
        }
    }
}
```

7. How Session Tracking Works



The session ID is commonly maintained through a cookie called **JSESSIONID**.

8. Http Session vs Cookies

HttpSession	**Cookies**
Data is mainly stored on server	Data is stored on client/browser
Uses a session ID	Stores cookie values
Suitable for login sessions	Suitable for preferences/small data
More suitable for sensitive session state	Client can modify cookie data
Session expires after timeout/invalidation	Cookie has its own expiration
Can store multiple attributes	Limited storage size

Conclusion

For an authentication system, **HttpSession is preferred** for maintaining server-side login state. Cookies are commonly used to carry the session identifier between the browser and server.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.
Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

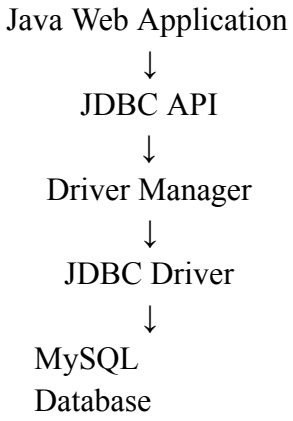
Answer:

JDBC E-Commerce Application

1. JDBC Architecture

JDBC stands for Java Database Connectivity.

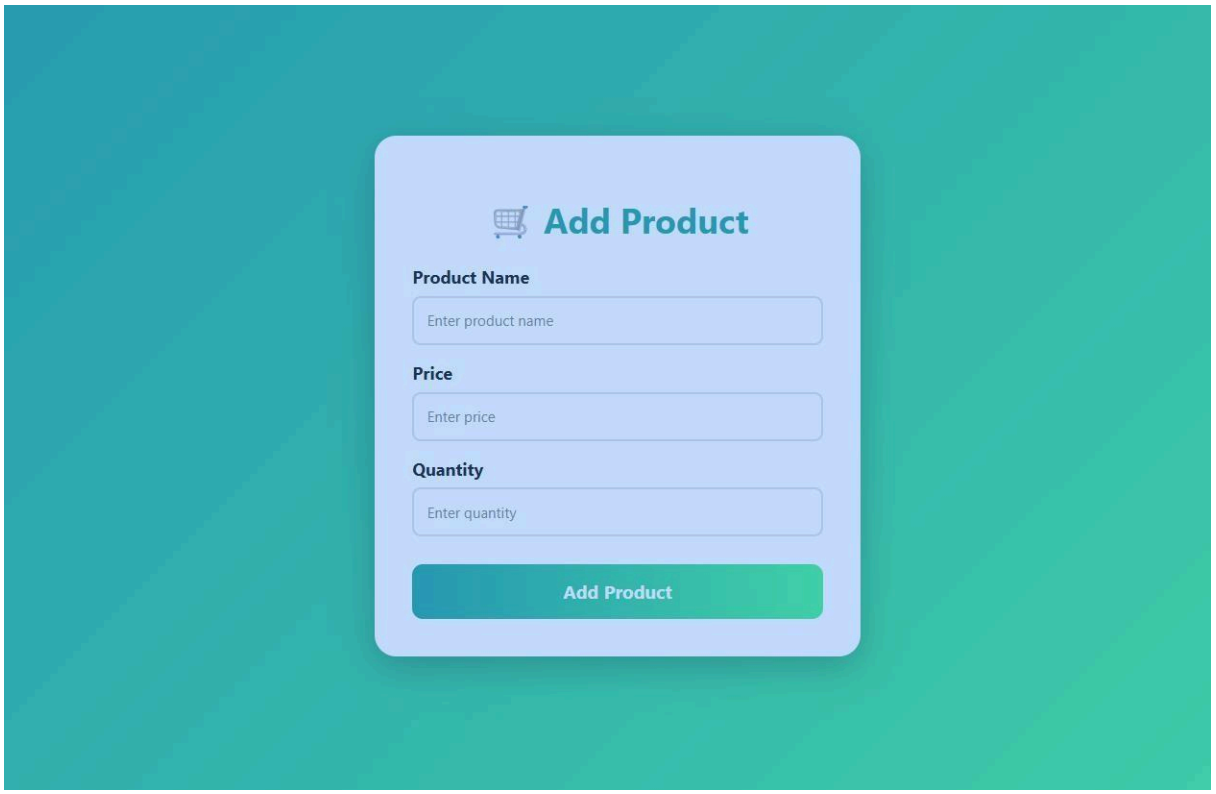
It allows Java applications to communicate with databases.



2. Database Creation

```
CREATE DATABASE ecommerce;

USE ecommerce;

CREATE TABLE product (
   id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(100),
   price DOUBLE,
   quantity INT
);
```

3. Colorful Product HTML Form



4. JDBC Database Connection

```
import java.sql.*;

public class DBConnection {

    public static Connection getConnection()
            throws SQLException {

        String url =
            "jdbc:mysql://localhost:3306/ecommerce";

        String username = "root";
        String password = "root";

        return DriverManager.getConnection(
            url,
            username
            ,
            password
        );
    }
}
```

Change "root" password according to your MySQL installation.

5. Insert Product Using Servlet

```
import java.io.*;
import java.sql.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class ProductServlet extends HttpServlet {

   protected void doPost(HttpServletRequest request,

                         HttpServletResponse response)
                         throws ServletException, IOException {

      response.setContentType("text/html");

      PrintWriter out = response.getWriter();

      String name =
         request.getParameter("name");

      double price =
         Double.parseDouble(
            request.getParameter("price"));

      int quantity =
         Integer.parseInt(
            request.getParameter("quantity"));

      String sql =
         "INSERT INTO product(name, price, quantity) " +
         "VALUES (?, ?, ?)";

      try (
         Connection con =
            DBConnection.getConnection();

         PreparedStatement ps =
            con.prepareStatement(sql)
      ) {

         ps.setString(1, name);
         ps.setDouble(2,
         price); ps.setInt(3,
         quantity);
```

```
            ps.executeUpdate();

            out.println("<html>");
            out.println("<body
style='font-family:Arial;background:#38ef7d;text-align:center;padding:50px'>");

            out.println("<div
style='background:white;padding:40px;border-radius:20px;width:450px;margin:auto'>");

            out.println("<h1 style='color:#11998e'>");
            out.println("✅ Product Added Successfully!");
            out.println("</h1>");

            out.println("<p>Product: " + name + "</p>");
            out.println("<p>Price: ₹" + price + "</p>");
            out.println("<p>Quantity: " + quantity + "</p>");

            out.println("</div>");
            out.println("</body>");
            out.println("</html>");

        } catch (SQLException e) {

            out.println("<h2
            style='color:red'>");
            out.println("Database Error: "
                    +
            e.getMessage());
            out.println("</h2>");
        }
    }
}
```

6. Retrieve Products

```
import java.io.*;
public class ProductListServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,

                         HttpServletResponse response)
                         throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();
```

```
        String sql =
            "SELECT * FROM product";

        out.println("<html>");
        out.println("<head>")
        ;
        out.println("<style>")
        ;
        out.println("body{font-family:Arial;background:linear-
gradient(135deg,#667eea,#764ba2);padding:40px;}");
        out.println("table{width:80%;margin:auto;background:white;border-
collapse:collapse;}");
        out.println("th{background:#5a4fcf;color:white;padding:15px;}");
        out.println("td{padding:12px;text-align:center;border-bottom:1px solid
        #ddd;}"); out.println("h1{text-align:center;color:white;}");
        out.println("</style>");
        out.println("</head>");

        out.println("<body>");

        out.println("<h1>🛒 Product List</h1>");

        out.println("<table>");

        out.println("<tr>");
        out.println("<th>ID</th>");
        out.println("<th>Product
        Name</th>");
        out.println("<th>Price</th>");
        out.println("<th>Quantity</th>");
        out.println("</tr>");

        try (
            Connection con =
                DBConnection.getConnection();

            Statement st =
                con.createStatement();

            ResultSet rs =
                st.executeQuery(sql)
        ) {

            while (rs.next()) {
```

```
            out.println("<tr>");

            out.println("<td>"
               + rs.getInt("id")
               + "</td>");

            out.println("<td>"
               + rs.getString("name")
               + "</td>");

            out.println("<td>₹"
               + rs.getDouble("price")
               + "</td>");

            out.println("<td>"
               + rs.getInt("quantity")
               + "</td>");

            out.println("</tr>");
         }

      } catch (SQLException e) {

         out.println("<tr><td
         colspan='4'>");
         out.println("Database Error: "
                  +
         e.getMessage());
         out.println("</td></tr>");
      }

      out.println("</table>");
      out.println("</body>")
      ;
      out.println("</html>");
   }
}
```

7. JDBC Steps

Remember these **5 important steps** for the exam:

Step 1 — Load JDBC Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Modern JDBC drivers can register automatically, but this is commonly shown in JDBC fundamentals.

Step 2 — Establish Connection

```
Connection con =
   DriverManager.getConnection(
      url, username, password);
```

Step 3 — Create Statement

```
PreparedStatement ps =
   con.prepareStatement(sql);
```

Step 4 — Execute SQL

For INSERT/UPDATE/DELETE:

```
ps.executeUpdate();
```

For SELECT:

```
ResultSet rs =
   ps.executeQuery();
```

Step 5 — Close Resources

```
rs.close();
ps.close();
con.close();
```

Using **try-with-resources**, as in the example above, automatically closes resources.

8. Exception Handling

JDBC operations can produce SQLException.

```
try {

   Connection con =
      DBConnection.getConnection();

   // Database operations

}
catch (SQLException e) {
```

```
    System.out.println(
        "Database Error: " + e.getMessage());

}
```

Code of Conduct:

I _____ (K.Deepika / 192371042) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Deepika

RUBRICS FOR CALCULATION:

Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale

					provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand